

Project Title: Bank Vault Attack Challenge (Phase 1)

Task:

Develop a secure, multi-user Command Line Interface (CLI) banking executable **using C**. This system will serve as the core interface on which other teams try to find exploits, representing the first layer of the Bank Vault architecture.

All money should be represented as 32bit signed integer.

Memory constraint is 32kB of RAM (including extra files you will be using to store data).

Account creation for each team is build time using some random generation.

The teams all start with 1000 points. (Leaderboard points will be a file managed by us)

Each day starts at 00:00 and ends at 23:59.

Username Max length is 32bytes

Functional Requirements:

1. User Management & Authentication

- **Multi-Account Support:** The system must support multiple distinct accounts.
- **Account Separation:** Strict logical isolation must be enforced between accounts; users should not be able to access or modify another user's data.
- **Authentication:** Critical operations (specifically withdrawals) must be secured via a Personal Identification Number (PIN), the PIN must be 6 digit hexadecimal input (eg. 123abc).
- **PIN Management:** Users must have the ability to change their PIN securely. For failed attempts maximum timeout is 3 second.

2. Banking Operations

- **Balance Query:** Users must be able to view their current account balance.
- **Deposit:** All teams will have 0 in other banks and each team can decide to deposit money from their points in leaderboard to use.
- **Withdrawals:** Users can withdraw funds, contingent upon successful PIN verification. This amount will be sent to your points. (The one in the leaderboard). A team can only withdraw how much ever they deposited + 2000 points in a phase. (ie if team despoited 200 points they can withdraw a max of 2200 and it is dynamic if that makes sense).
- **Loans:** Implement a facility for users to request and receive loans directly from the Admin account. (Also check loan status and current interest to be paid). Loan of maximum 500 points can be taken. Interest of 3% per day.
- **Transactions:** Add a PIN protected feature to transact either using credit card or your bank account.
- **Transaction History:** All bank operations done (newest 50) using this account should be readable only by the person who has access to the account. (Format: Operation, Amount, Time + Relevant information as required.)

3. Fixed Deposit (FD) Module

- **Create FD:** Users can transfer a selected amount from their main balance into a Fixed Deposit.
 - *Validation:* The system must verify that the user has sufficient funds before creation.
- **Interest Calculation:** The FD balance must multiply based on a 2.5% per day.
- **Break FD:** Users must be able to liquidate their FD, transferring the principal plus accumulated interest back into their main account balance.

4. Administrative Privileges

- **The ADMIN Account:** A superuser account ("ADMIN") must be implemented.

- **Capabilities:** This account is unrestricted and must possess the ability to perform any operation on the system without needing a pin.
 - Admin shouldn't reset pin of other teams. Can withdraw from other teams tho ;)
-

Technical & Security Constraints:

1. Sensitive Data Handling

- **Credit Card Integration:** The system must handle and store Credit Card Numbers (PAN) and CVV codes associated with accounts. Card "month" is 5 days and interest of 2%. Card limit of 500 points.

2. Delivery Platform

- The final deliverable must be a compiled **C executable**.
-

Deliverables

- Source code in C.
- Compiled CLI Executable meeting all functional and security constraints listed above.
- [README.md](#) of how to use your bank.